Module 9 Individual Assignment: No Code Programming Environments

Laws Smith
SWE 632 - Spring 2026

Application Domain: Home Automation

I've been running Home Assistant for a while now and it's one of those projects I genuinely enjoy tinkering with. I have automations that coordinate sleep routines between multiple people, adjust lights based on who's home and what time of day it is, manage climate control when it's hot out, and even run weekly system updates that loop through every addon. At this point my setup has real complexity to it, with variables, conditionals, loops, and reusable routines. So when this assignment asked us to design a no-code programming environment, home automation was an easy pick. I already know where the pain points are.

The biggest one is how you actually build automations. Home Assistant gives you two options: a visual drag-and-drop editor or raw YAML code. The visual editor is great for simple stuff, but it falls apart with complex logic. At that point you switch to YAML, and once you do, you lose the visual view entirely. Switching back often breaks things. You're stuck choosing one or the other, and neither is great on its own.

AutomateHome is my take on fixing this. It's a visual programming environment where you build automations by dragging and connecting blocks, but with one difference that changes everything: the code view is always right there next to the canvas, updating in real time as you work.

The Big Idea: Side-by-Side Code Sync

As you drag a block onto the canvas, the YAML on the right updates instantly. Click a block and the corresponding lines highlight in the code. Edit the YAML directly and the blocks rearrange to match. This is powerful for a few reasons.

It helps you learn the code. Most people start with the visual editor because it's approachable. But as their automations grow, they eventually need to understand the code. Instead of a hard cutover from visual to code, the side-by-side view lets you watch the code take shape as you drag blocks around. You start to recognize patterns. "Oh, that's what a condition looks like in YAML." Over time, you naturally pick up the syntax without ever sitting down to study it.

It gives you code-level control when you need it. Sometimes the visual editor is faster. Sometimes typing a line of YAML is faster. With the sync view, you don't have to choose. Drag a block to set up the

basic structure, then click into the YAML to fine-tune a specific value. Both sides stay in sync, so there's no risk of one overwriting the other.

It makes debugging easier. When an automation isn't working, being able to see both the visual flow and the underlying code at the same time helps you figure out what went wrong. You can trace through the flow visually while checking the actual values in the code.

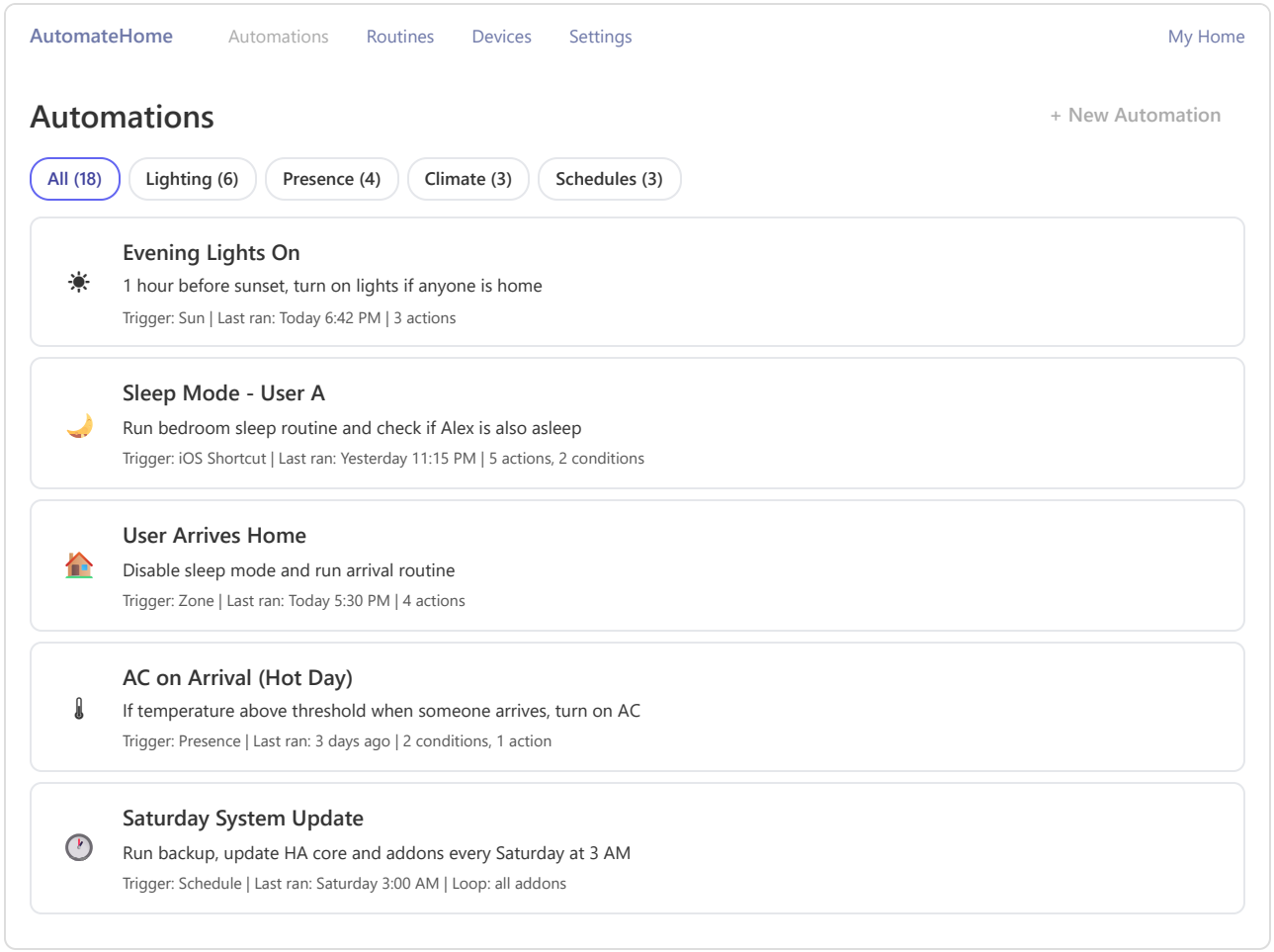
This is Shneiderman's direct manipulation in action: continuous representation (the automation is always visible in both forms), physical actions instead of syntax (drag blocks to build logic), and immediate feedback (every change shows up instantly in both views).

Programming Constructs

Construct	How It Works	Example
Variables	Parameter blocks and helper set/get blocks	ac_temp_threshold = 70, "Lights On Time" = ON
Conditionals	If/Else block splits into YES/NO visual paths	Is Alex asleep? YES: master sleep. NO: wait
Loops	Dashed border wraps repeated actions	For each addon: update, wait 30s
Functions	Routines with parameters, reusable from palette	"Bedroom Sleep" with person_name param

Screen 1: Automation List

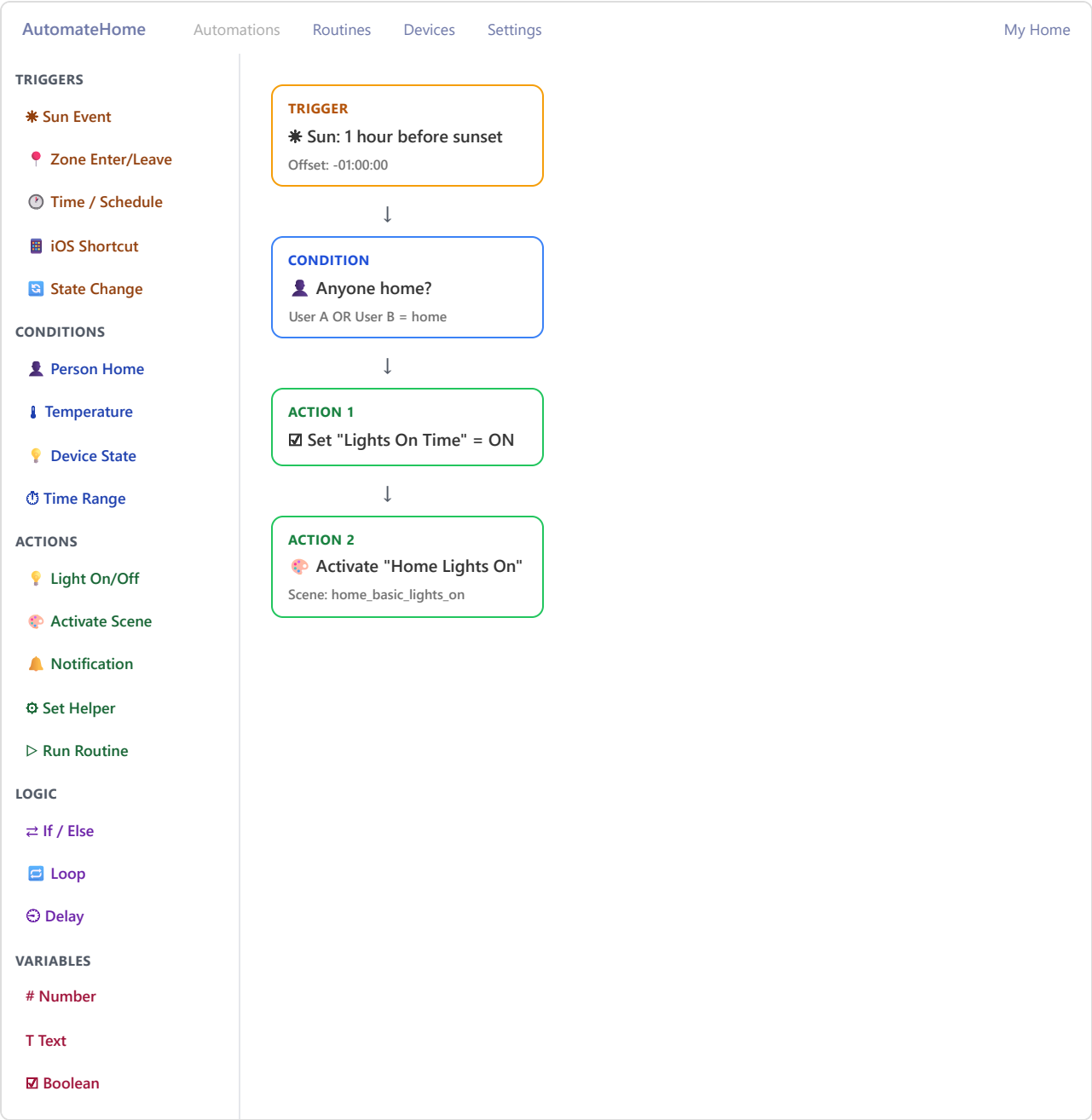
The main dashboard showing all automations at a glance. Each card shows the name, description, when it last ran, how many actions and conditions it has, and a toggle to enable or disable it. Filter chips at the top narrow by category.



Each card gives enough context to understand the automation without opening it. The "last ran" timestamp helps catch automations that should have fired but didn't. The toggle lets you disable something temporarily without editing or deleting it.

Screen 2: Visual Canvas

The core editing experience. A block palette on the left has draggable blocks organized by type. You drag them onto the canvas and they snap into a top-to-bottom flow connected by arrows. Click any block to edit its properties inline. This shows the "Evening Lights On" automation.

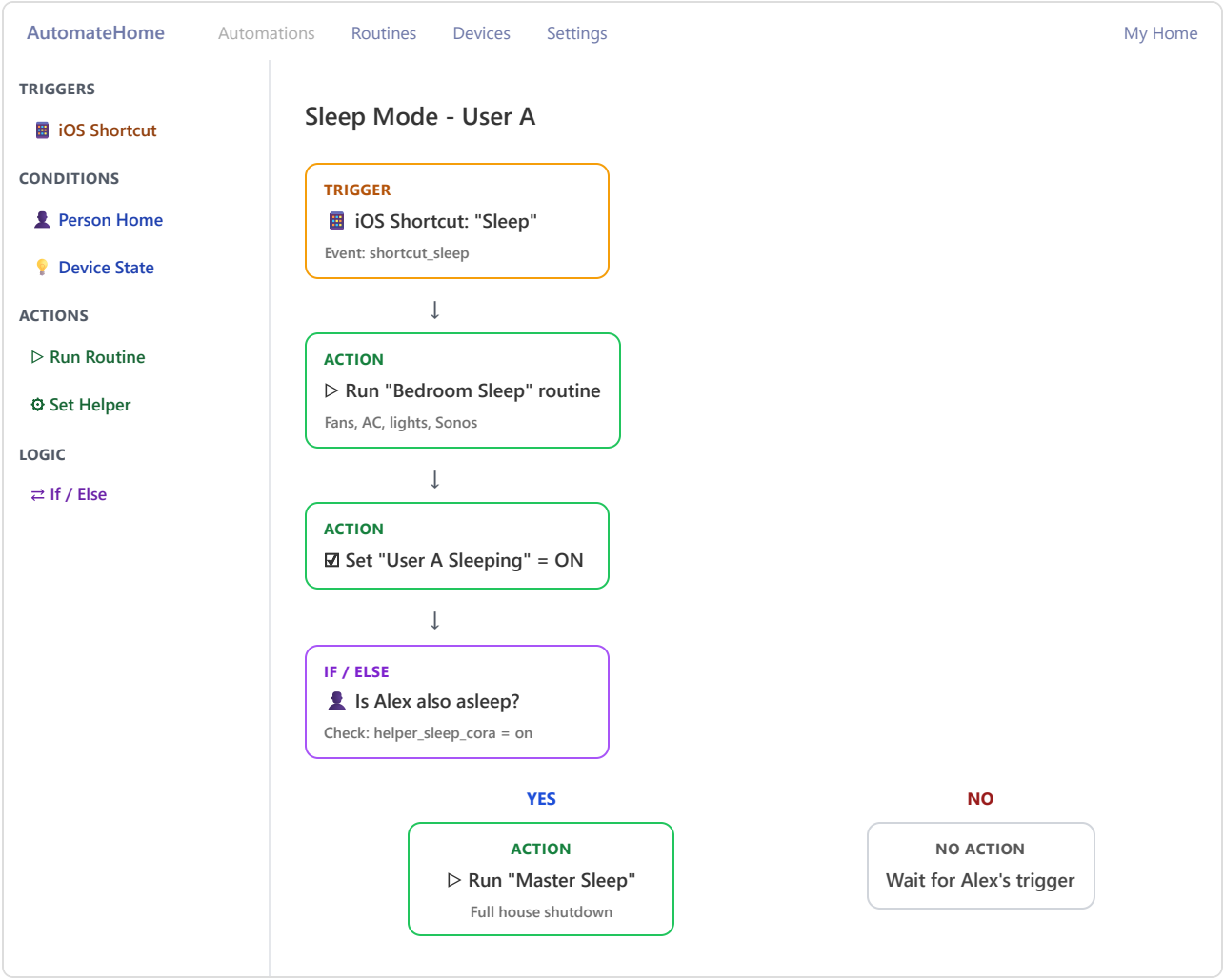


Blocks are color-coded by type: yellow for triggers, blue for conditions, green for actions, purple for logic, and pink for variables. This makes the flow scannable at a glance. The palette on the left contains every available block type organized by category, and blocks are added by dragging them onto the canvas.

The "Set Helper" action block demonstrates the variable construct. Helper blocks store boolean, number, or text values that other automations can read. Here, "Lights On Time" is a boolean that other automations check to decide whether it's evening.

Screen 3: Conditionals

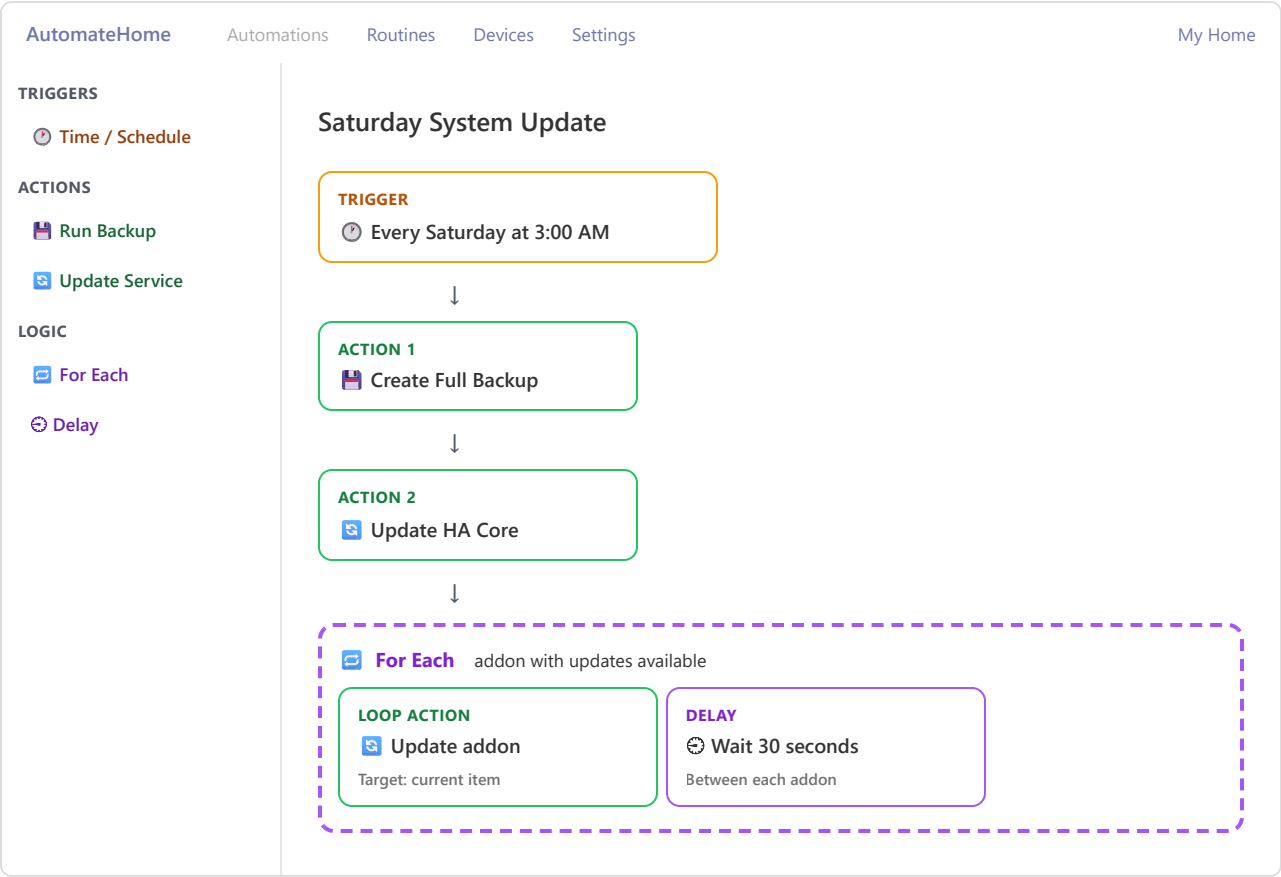
This shows the Sleep Mode automation, where conditional logic renders as a forking path. After running the bedroom routine and setting a sleep flag, the flow branches: if Alex is also asleep, run the full house shutdown. If not, do nothing and wait for her own sleep trigger.



In the current Home Assistant editor, this kind of multi-person coordination logic shows up as a flat list of conditions that's hard to follow. Showing it as a forking path makes the logic obvious. You can see both outcomes at once and understand the relationship between them. If/Else blocks can also be nested inside branches to create multi-level decision trees.

Screen 4: Loops and Schedules

The Saturday System Update automation demonstrates loops. After backing up and updating the core, a "For Each" block iterates through every addon with available updates. The loop is a dashed border wrapping the repeated actions.

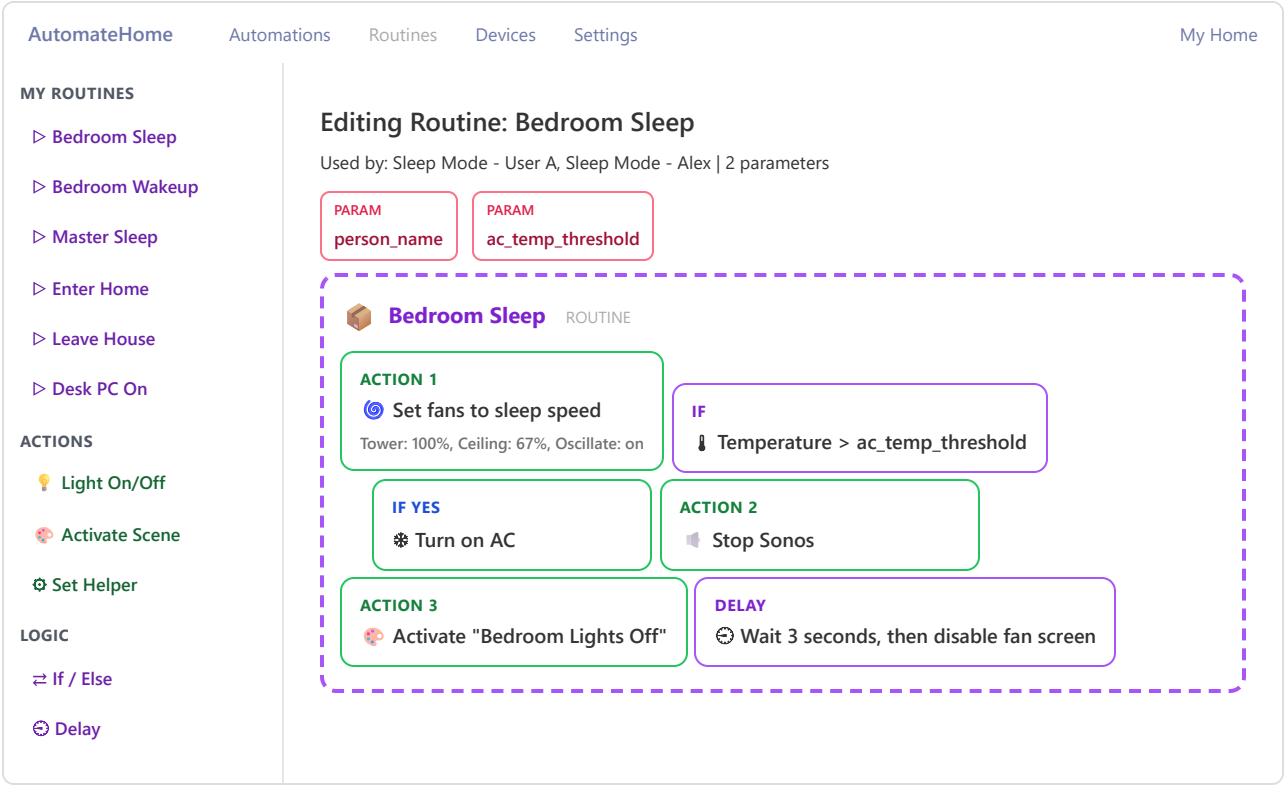


Loops are one of the things the existing HA visual editor can't represent. You end up writing Jinja2 templates in YAML. Here, you just drag a "For Each" block and drop actions inside it. The dashed border makes it clear which actions repeat and which don't. The loop variable ("current item") is available to everything inside the border.

The schedule trigger supports daily, weekly, monthly, and custom cron expressions, configured through a calendar picker instead of typed syntax.

Screen 5: Routines (Functions)

Routines are reusable groups of actions that work like functions. "Bedroom Sleep" accepts two parameters so the same logic works for both residents with different temperature preferences. Routines appear in the palette and can be dragged into any automation.



Parameters are shown as pink blocks at the top of the routine. Inside, actions can reference these parameters. The "If Temperature > ac_temp_threshold" condition uses the parameter value, so one person can set it to 70 and the other to 72 when calling the same routine. If you update the routine, the change automatically applies everywhere it's used.

Screen 6: Code Sync and Live Preview

The split view that solves the core problem. Visual canvas on the left, YAML on the right, always in sync. Click a block to highlight the corresponding code. Below the canvas, a simulation panel lets you test automations by adjusting device states.

The condition block is selected (highlighted border on the left), and the corresponding YAML lines are highlighted on the right. Editing either side updates the other. The green "Synced" badge confirms both views match. You can drag blocks for structure, then fine-tune values in the code, without ever worrying about one overwriting the other.

The simulation panel at the bottom lets you drag a time slider and see what would happen. Green badges show which actions would fire. You can test your automation before it ever runs for real, which matters when you're controlling physical devices in your home.

AI Disclosure: AI tools were used to generate the HTML wireframes. The application domain, pain points, and design decisions come from my personal experience running Home Assistant.